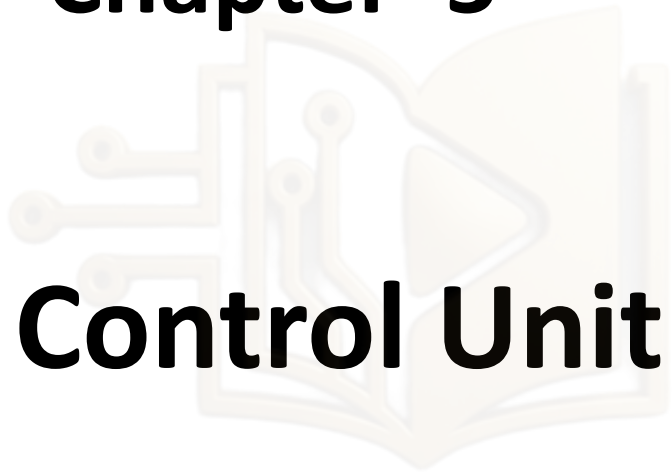


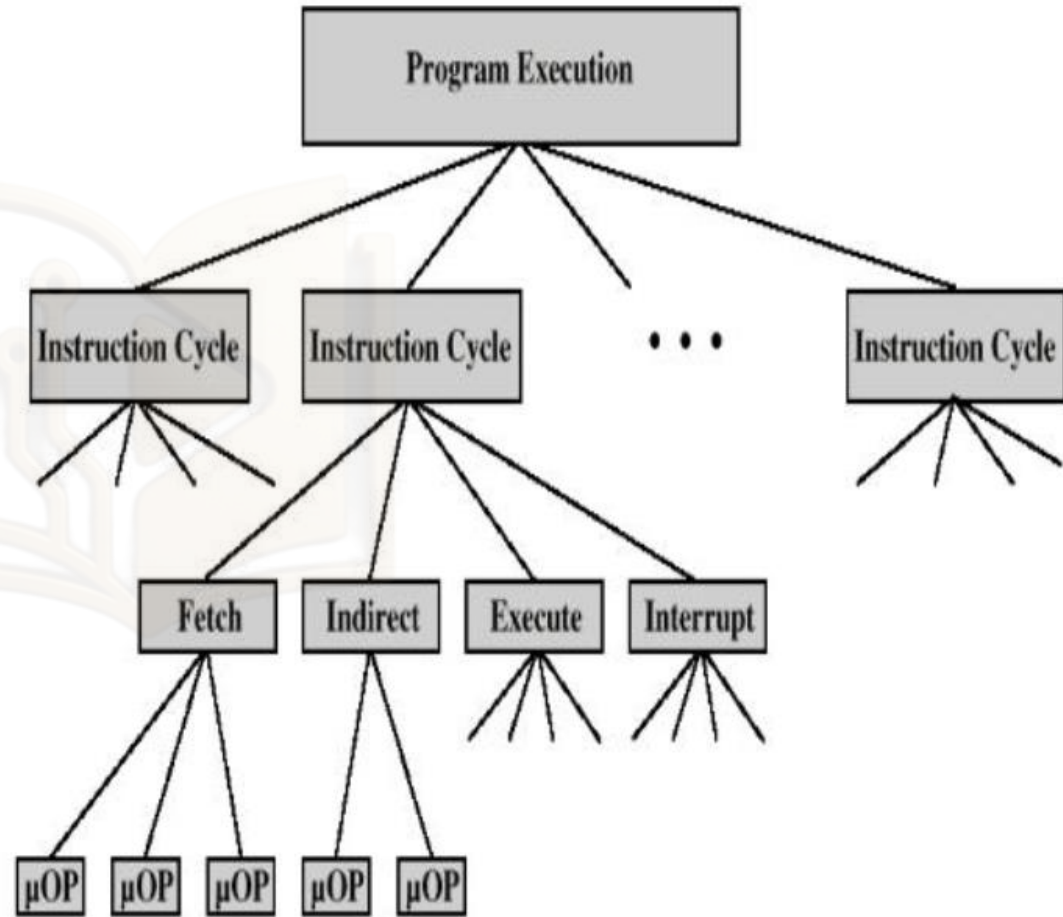
Chapter 5

Control Unit



Micro-operations

- Are the functional, or atomic, operations of a processor.
- A single *micro-operation* generally involves a transfer between registers, transfer between registers and external bus, or a simple ALU operation.



Micro-operations and the Clock

- Each clock pulse defines a time unit, which are of equal duration.
- *Micro-operations* are performed within this time unit.
- If multiple *micro-operations* do not interfere with one another then grouping of micro-operations can be performed within one time unit. Here, MDR == MBR (Memory buffer/data register)
- Grouping can be performed as long as;
 - Proper sequence of events are followed
 - PC → MAR must be done first in order for MEMORY → MDR
 - Conflicts are avoided
 - MEMORY → MDR can not be in the same time unit as MDR → IR

The Fetch Cycle

- Consists of three time units and four *micro-operations*.
- Each *micro-operation* involves the movement of data into or out of a register.

$$t_1 : PC \rightarrow MAR$$

$$t_2 : MEMORY \rightarrow MDR$$

$$t_3 : PC + 1 \rightarrow PC$$

$$MDR \rightarrow IR$$

The Indirect Cycle

- Occurs if the instruction specifies an indirect address.
- Consists of three time unit and three *micro-operations*.
- Data is transferred to the MAR from the IR, which is used to fetch the address of the operand, the IR is then updated from MDR so it contains a direct address rather than indirect.

$$t_1 : IR \rightarrow MAR$$

$$t_2 : MEMORY \rightarrow MDR$$

$$t_3 : MDR \rightarrow IR$$

The Interrupt Cycle

- Occurs if any enabled interrupts have occurred at the completion of the execute cycle.
 - The contents of the PC are transferred to the MDR, so that they can be saved for return from the interrupt.
 - MAR is loaded with the address at which the contents of the PC are to be saved
 - PC is loaded with the address at the start of the interrupt routine.
 - Final step is to store the MDR into MEMORY.

$t_1 : PC \rightarrow MDR$

$t_2 : \text{Saved Address} \rightarrow MAR$

Routine Address $\rightarrow PC$

t_3

$t_4 : MDR \rightarrow MEMORY$

The Execute Cycle

- Execute cycle is not as predictable as other cycles (fetch, indirect, or interrupt).
- Number of time units and micro-operations varies for every execution cycle.

Example; ADD R1, X

- The following execute cycle adds the contents of the location X to register R1.

$t_1 : \text{IR} \rightarrow \text{MAR}$

$t_2 : \text{MEMORY} \rightarrow \text{MDR}$

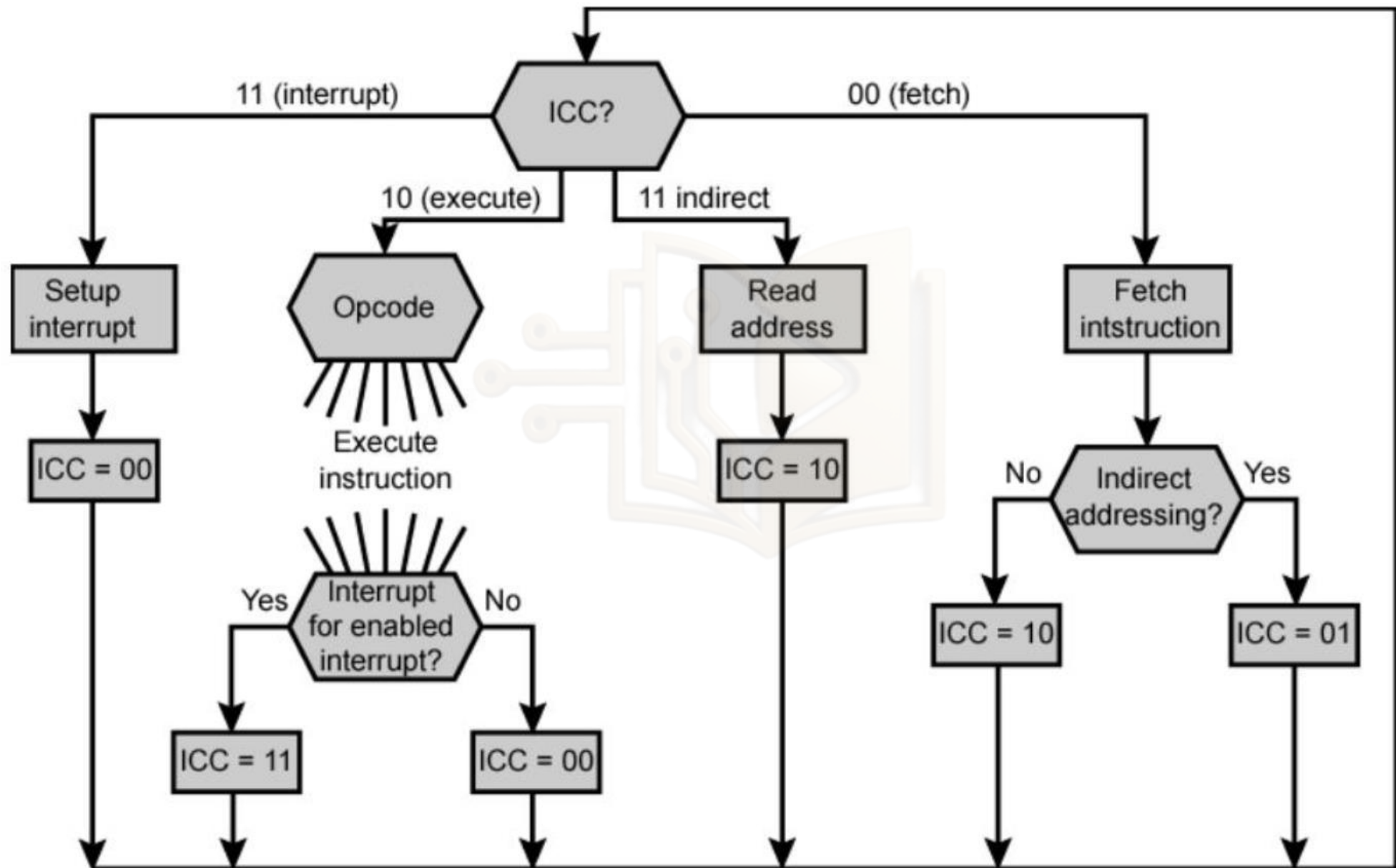
$t_3 : \text{MDR} + \text{R1} \rightarrow \text{R1}$

Instruction Cycle

- Each phase decomposed into sequence of elementary micro-operations (fetch, indirect, and interrupt cycles)
- Execute cycle
 - One sequence of micro-operations for each opcode
- Need to tie sequences of micro-operations together
- Assume new 2-bit register
 - Instruction cycle code (ICC) designates which part of cycle the processor is in:

00: Fetch	10: Execute
01: Indirect	11: Interrupt

Flowchart for Instruction Cycle



Functional Requirements

- Define basic elements of processor
- Describe micro-operations processor performs
- Determine functions control unit must perform

Basic Elements of the Processor

ALU

External data paths

Control Unit

Registers

Internal data paths

Types of Micro-operation

- Transfer data between registers
- Transfer data from register to external
- Transfer data from external to register
- Perform arithmetic or logical operations

Functions of Control Unit

- *Sequencing*
Causes the processor to step through a series of micro-operations
- *Execution*
Causes the performance of each micro-operation
- *This is done using Control Signals*

Characterization of the control unit

- Define the basic elements of the processor.
- Describe the micro-operations that the processor performs.
- Determines the function that the control unit must perform to cause the micro-operations to be performed.

Control Unit Functions

- **Sequencing:** control unit causes the processor to step through a series of micro-operations in sequence, based on the program being executed
- **Execution:** control unit causes each micro-operation to be performed

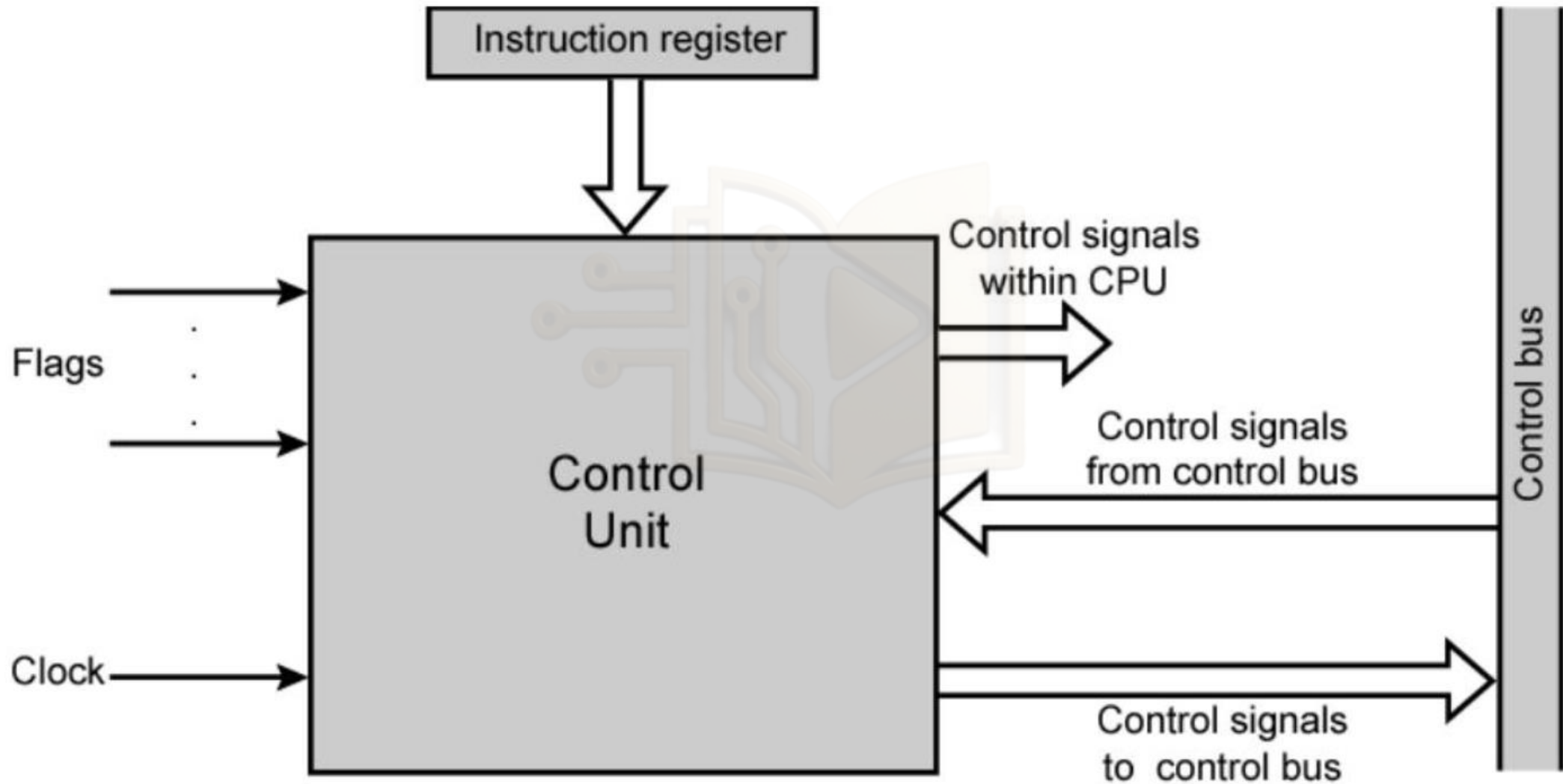
Control Unit Inputs

- **Clock:** control unit causes one micro-operation (or set of simultaneous micro-operations)
- **Instruction register:** opcode of the current instruction is used to determine which micro-operations to perform during the execute cycle
- **Flags:** needed by the control unit to determine the status of the processor and the outcome of previous ALU operations
- **Control signals from control bus:** Provides signals to the control unit (interrupt, acknowledgement)

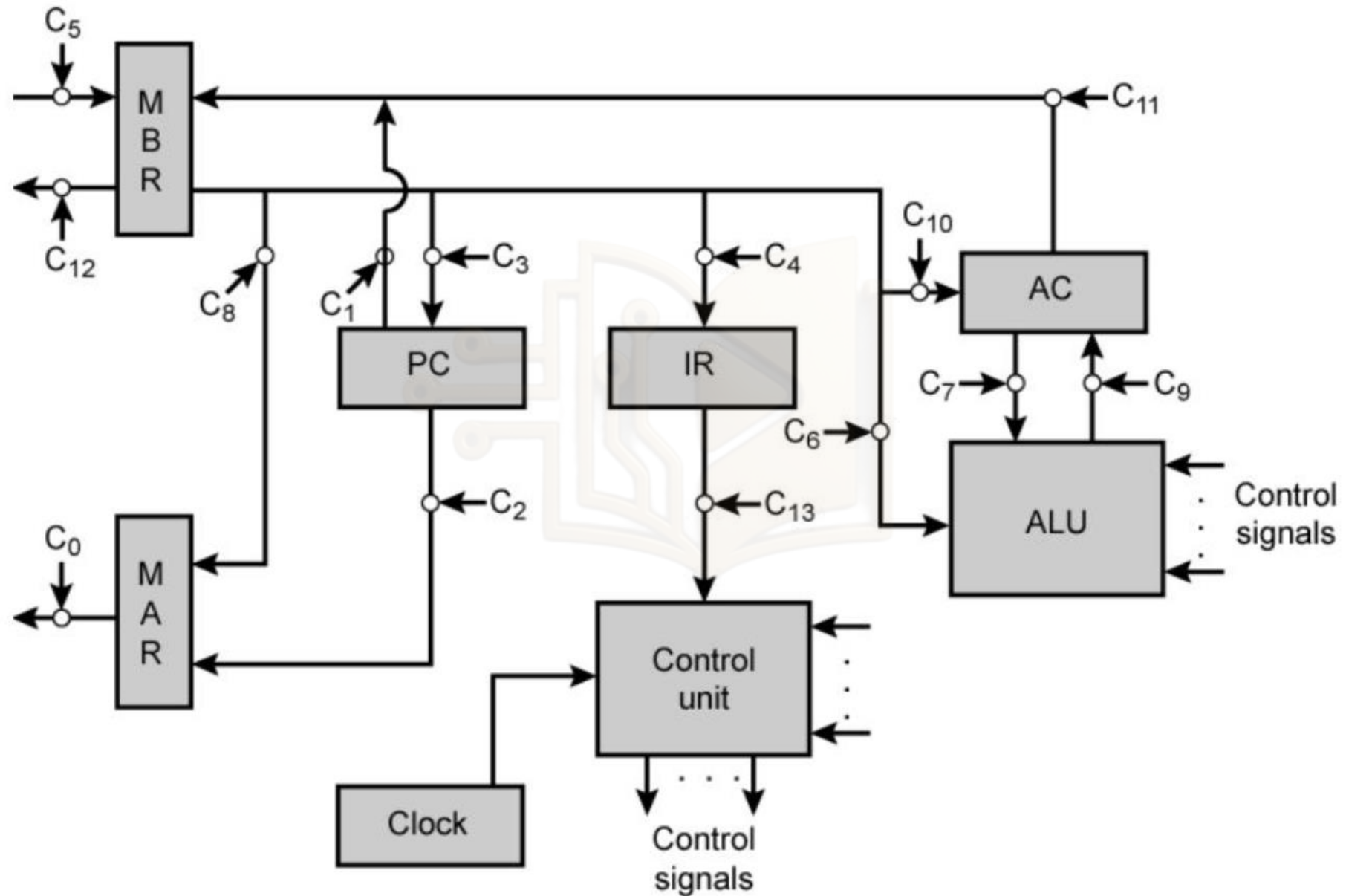
Control Unit Outputs

- **Control signal within the processor:** (type 1): cause data to be moved from one register to another.
(type 2): activate ALU function.
- **Control signals to control bus:** (type 1): control signals to memory. (type 2): control signals to the I/O modules.

Model of Control Unit



Data Paths and Control Signals



Micro-operations and control signals

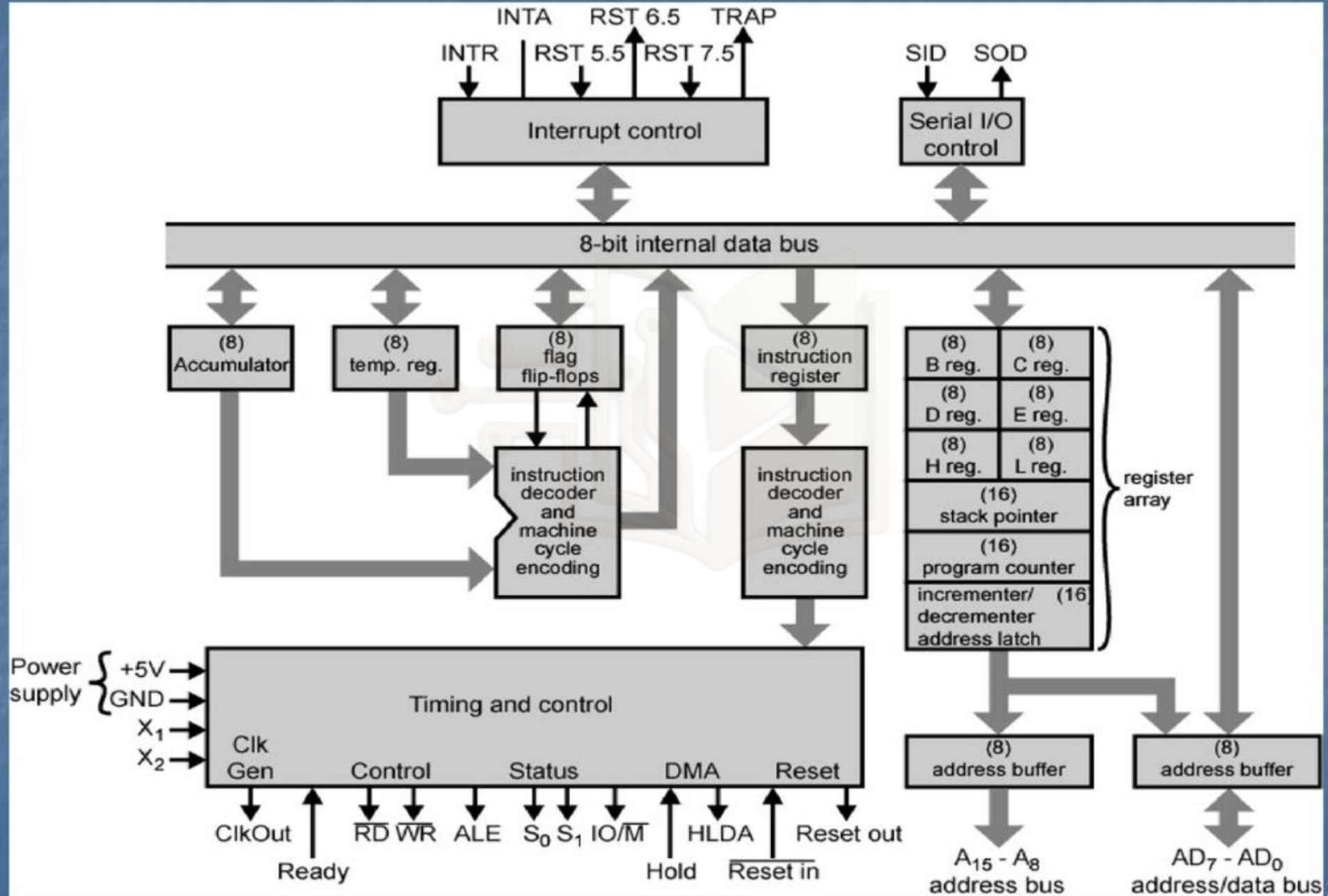
Micro-operations	Timing	Active Control Signals
Fetch:	$t_1: \text{MAR} \leftarrow (\text{PC})$	C_2
	$t_2: \text{MBR} \leftarrow \text{Memory}$ $\text{PC} \leftarrow (\text{PC}) + 1$	C_5, C_R
	$t_3: \text{IR} \leftarrow (\text{MBR})$	C_4
Indirect:	$t_1: \text{MAR} \leftarrow (\text{IR}(\text{Address}))$	C_8
	$t_2: \text{MBR} \leftarrow \text{Memory}$	C_5, C_R
	$t_3: \text{IR}(\text{Address}) \leftarrow (\text{MBR}(\text{Address}))$	C_4
Interrupt:	$t_1: \text{MBR} \leftarrow (\text{PC})$	C_1
	$t_2: \text{MAR} \leftarrow \text{Save-address}$ $\text{PC} \leftarrow \text{Routine-address}$	
	$t_3: \text{Memory} \leftarrow (\text{MBR})$	C_{12}, C_W

C_R = Read control signal to system bus.
 C_W = Write control signal to system bus.

PROCESSOR ORGANIZATION

- Organization is how features are implemented
 - Control signals, interfaces, memory technology.
 - e.g. Is there a hardware multiply unit or is it done by repeated addition

Intel 8085 CPU Block Diagram



Internal Organization

- Usually a single internal bus
- Using single bus simplifies & saves space
- Gates control movement of data onto and off the bus
- Control signals control data transfer to and from external systems bus
- Temporary registers needed for proper operation of ALU

- To execute the instructions, the processor must have some means of generating **control signals** needed in the proper sequence.
- Computer designers use a wide variety of technology to solve this problem.
- This approach fall into two categories:
 - i. Hardwired Control
 - ii. Microprogrammed control

Control Unit Design Approaches

1. Hardwired:

- The control logic is wired into logic circuits

2. Microprogrammed

- Here the control logic is coded in to a low level program

Hardwired Implementation (Block diagram)

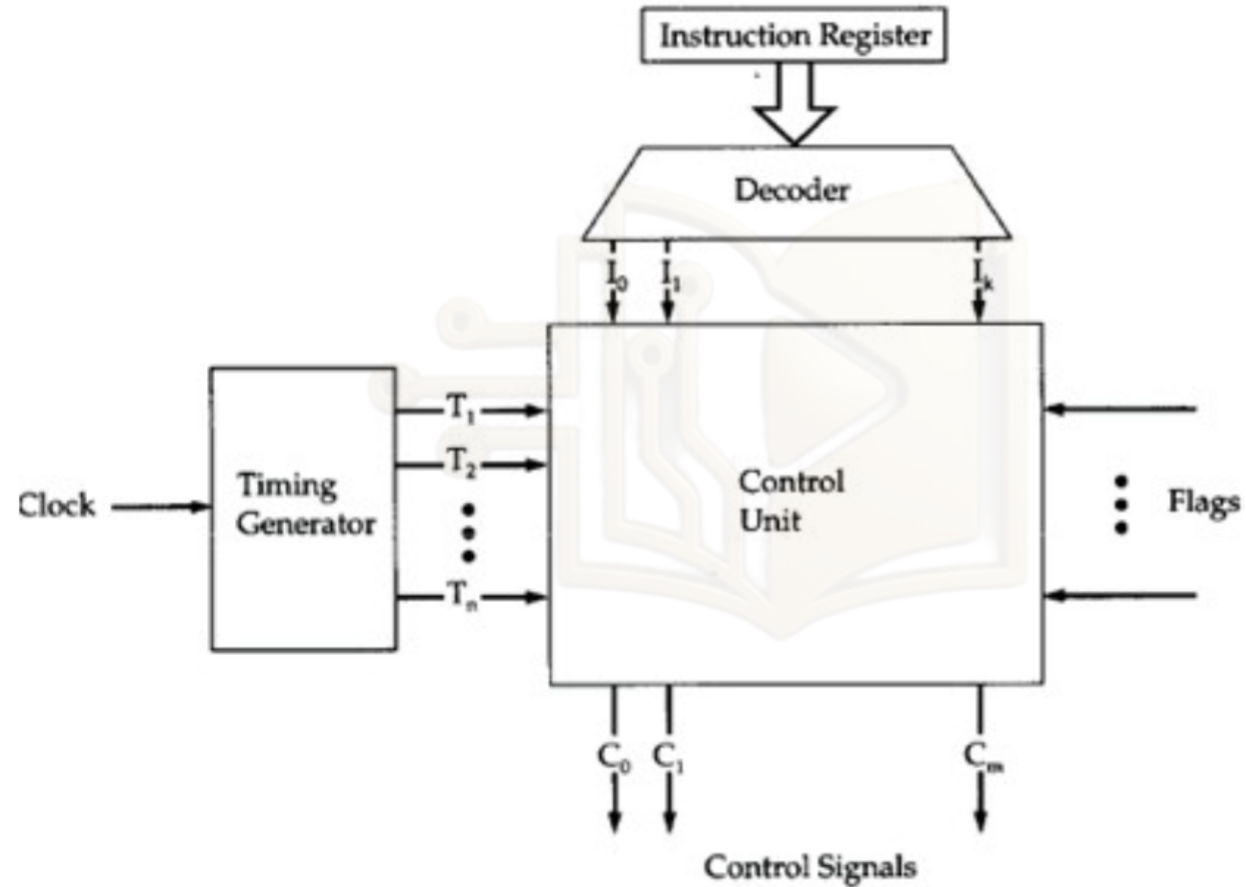


Figure: Hardwired control unit with decoded inputs

Hardwired Implementation (1)

- Control unit inputs
- Flags and control bus
 - Each bit means something
- Instruction register
 - Op-code causes different control signals for each different instruction
 - Unique logic for each op-code
 - Decoder takes encoded input and produces single output
 - n binary inputs and 2^n outputs

Hardwired Implementation (2)

- Clock
 - Repetitive sequence of pulses
 - Useful for measuring duration of micro-ops
 - Must be long enough to allow signal propagation
 - Different control signals at different times within instruction cycle
 - Need a counter with different control signals for t_1 , t_2 etc.

Hard Wired Control Unit

- The Cycles (Fetch, Indirect, Execute, Interrupt) are constructed as a State Machine
- The Individual instruction executions can be constructed as State Machines
 - Common sections can be shared. There is a lot of similarity
- One ALU is implemented. All instructions share it

Problems With Hard Wired Designs

- Complex sequencing & micro-operation logic
- Difficult to design and test
- Inflexible design
- Difficult to add new instructions
- **Disadvantage:**
 - Complex sequencing & micro-operation logic
 - Difficult to design and test
 - Inflexible design
 - Difficult to add new instructions
- **Advantage:**
 - Faster

In Hardwired implementation, the CU is essentially combinatorial circuit. Its i/p logic signals are transformed into a set of logic signals, which are the control signals. It is implemented through the use of finite no. of gates. Its design uses a fixed architecture. It requires change in the wiring if the instruction set is modified or change. This architecture is performed as RISC (Reduced Instruction Set Computer). Control Unit I/p:- It is given a decoded o/p from IR. Control Unit logic:- $PQ = 00$ Fetch 01 Indirect 10 Execute 11 Interrupt Then the Boolean expression that defines $C5$: $P'.Q'.T2' + P'.Q.T2 + PQ'.T3$ (That is control signals $C5$ will be asserted during the second time Unit and both fetch and indirect cycles.)

This expression is not complete $C5$ is also needed during Execute Cycle.

Let assume, only three instruction that read memory : LDA, ADD and AND now $C5$ can be defined as: $C5: P'.Q'.T2' + P'.Q.T2 + PQ'.T3 + P.Q'.(LDA + ADD + AND).T2$

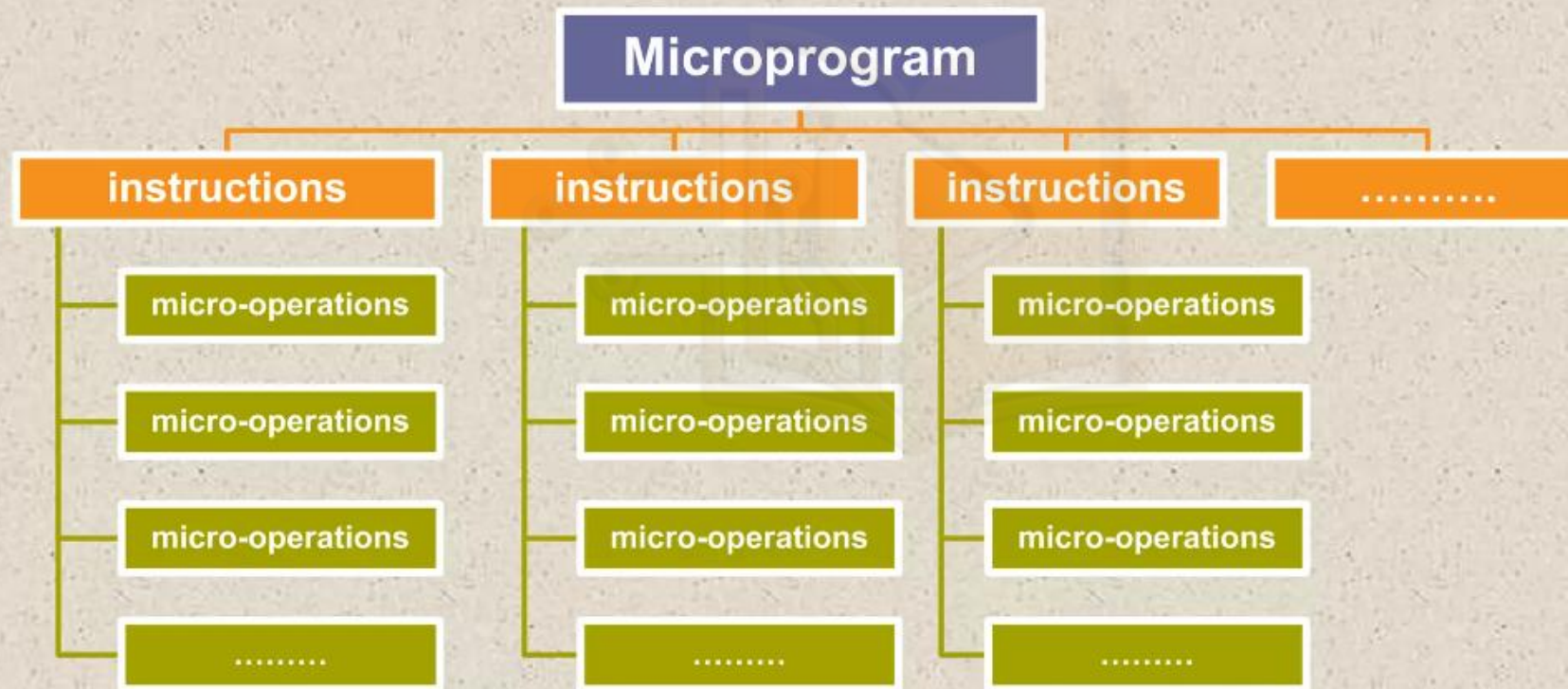
[here above expression is obtained by taking random specific example, expression depend on taken example]

Micro programmed Control Unit

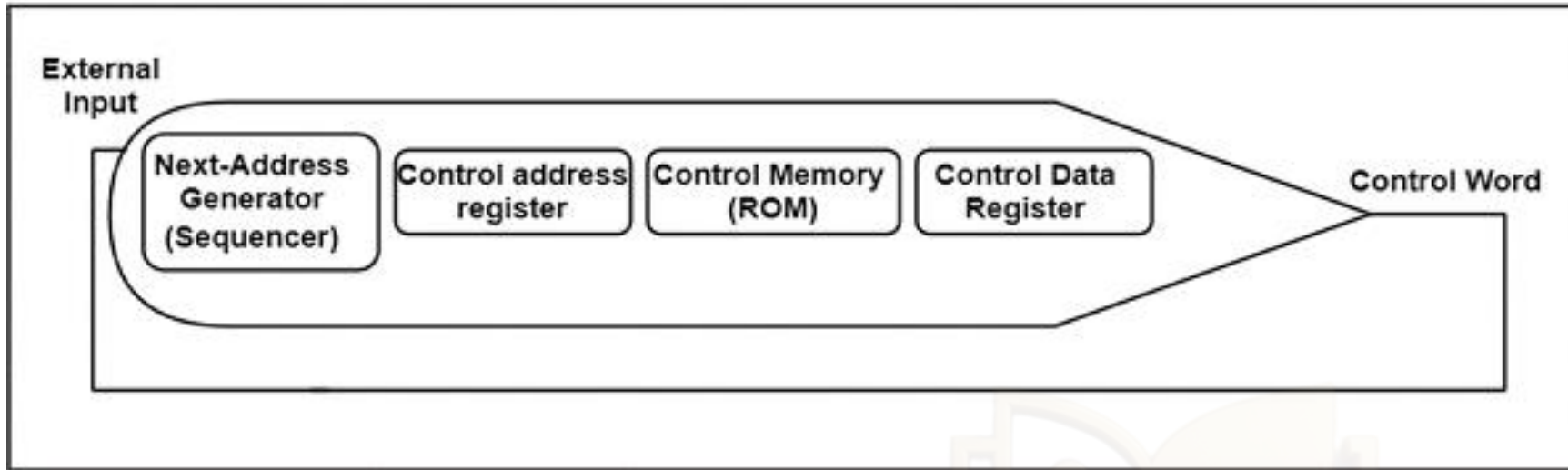
- **An alternative to a hardwired control unit is a microprogrammed control unit(specified by a microprogram)**
- **Microprogrammed control unit**
 - Sequencing through microinstructions
 - Generating control signals to execute each microinstruction
- **A control signals are used to cause register transfers and ALU operations**
- **An approach to control unit design that was organized and systematic and avoid the complexities of a hardwired implementation**



Microprogrammed Control



Micro Programmed Control Organization



There are the following steps followed by the microprogrammed control are –

- It can execute any instruction. The CPU should divide it down into a set of sequential operations. This set of operations are called microinstruction. The sequential micro-operations need the control signals to execute.
- Control signals saved in the ROM are created to execute the instructions on the data direction. These control signals can control the micro-operations concerned with a microinstruction that is to be performed at any time step.
- The address of the microinstruction is executed next is generated.
- The previous 2 steps are copied until all the microinstructions associated with the instruction in the set are executed.

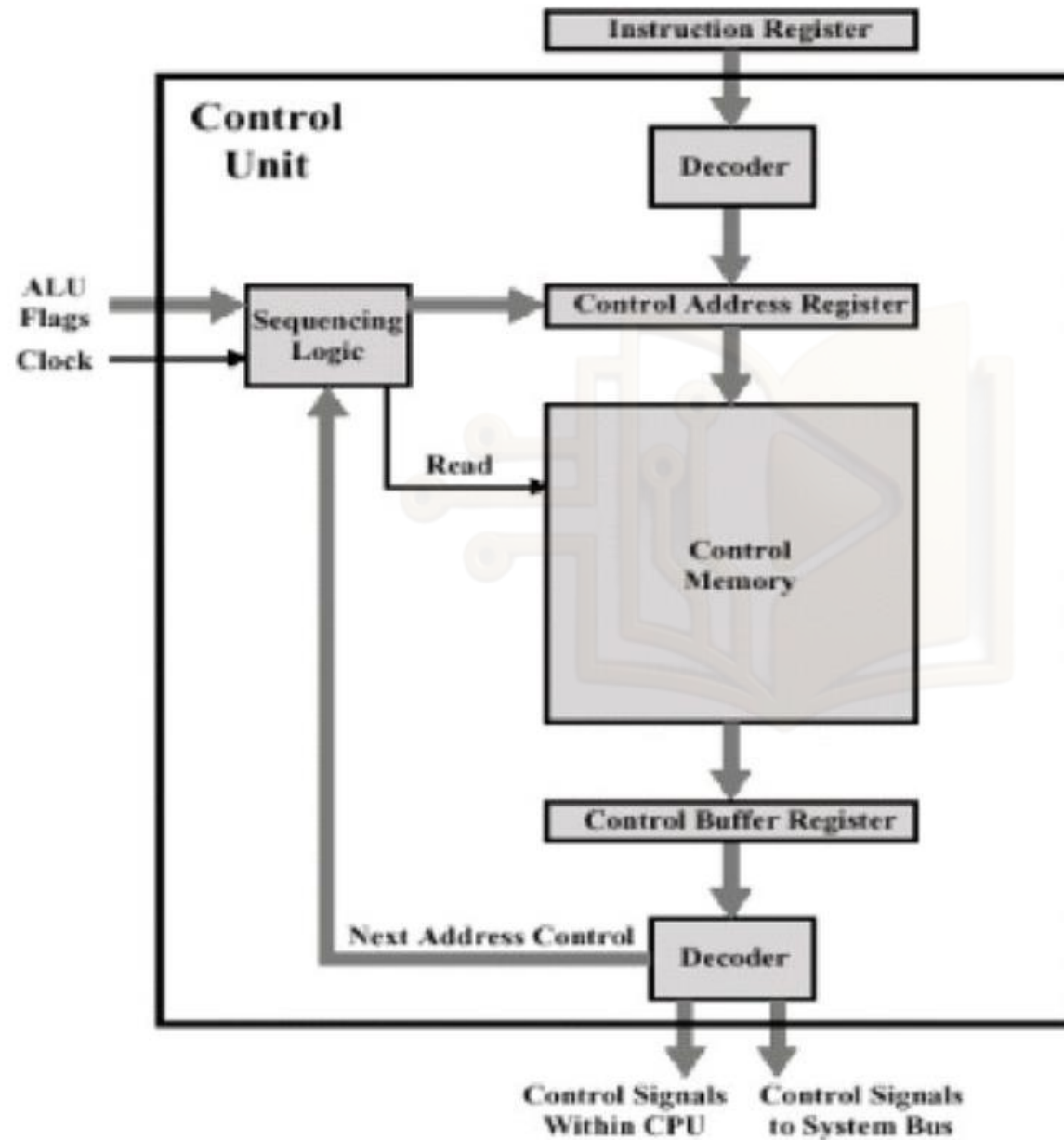
The address that is supported to the control ROM originates from the micro counter register. The micro counter received its inputs from a multiplexer that chooses the output of an address ROM, a current address incrementor, and an address that is saved in the next address field of the current microinstruction.

COMPARISON BETWEEN HARDWIRED AND MICROPROGRAMMED CONTROL UNIT

ATTRIBUTE	HARDWIRED CONTROL UNIT	MICRO-PROGRAMMED CONTROL UNIT
SPEED	FAST	SLOW
COST OF IMPLEMENTATION	MORE	CHEAPER
FLEXIBILITY	NOT FLEXIBLE, DIFFICULT TO MODIFY FOR NEW INSTRUCTION	FLEXIBLE, NEW INSTRUCTIONS CAN BE ADDED
ABILITY TO HANDLE COMPLEX INSTRUCTION	DIFFICULT	EASIER
DECODING	COMPLEX	EASY
APPLICATION	RISC MICROPROCESSOR	CISC MICROPROCESSOR
INSTRUCTION SET SIZE	SMALL	LARGE
CONTROL MEMORY	ABSENT	PRESENT
CHIP AREA REQUIRED	LESS	MORE

Microprogrammed Control Unit

Basic Concepts



- **To implement a control unit as an interconnection of basic logic elements is no easy task**
- **An alternative, which is quite common in contemporary CISC processors, is to implement a microprogrammed control unit.**
- **Microprogramming language**
- **Microinstruction**
 - A sequence of instructions is a microprogram, or firmware
 - Easier to design in firmware than hardware
 - More difficult to write a firmware than a software

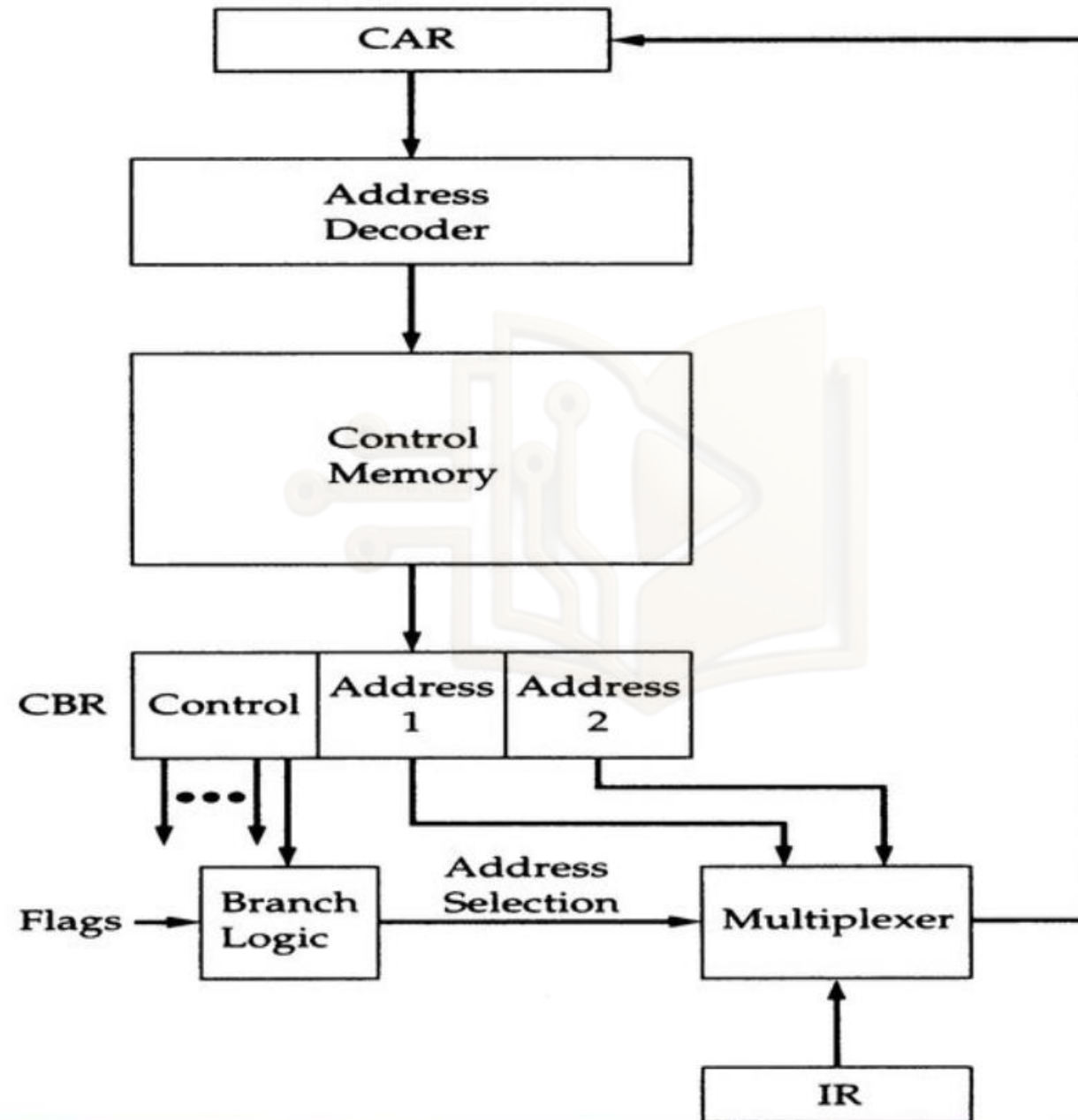
- **All that the control unit is allowed to do is generate a set of control signals.**
- **This condition(either on and off) can be represented by a binary digit for each control line.**
- **So we construct a control word in which each bit represents one control line.**

- **The two basic task(microprogrammed control unit)**
 - **Microinstruction sequencing** : Get the next microinstruction from the control memory
 - **Microinstruction execution** : Generate the control signals needed to execute the microinstruction
- **Design Consideration**
 - The address of the next microinstruction to be executed
 - Determined by instruction register – occurs once per instruction cycle
 - Next sequential address – most common
 - Branch – necessary part of a microprogram

- **Condition flags, the contents of the instruction register and a control memory address must be generated for the next microinstruction.**
- **Based on the format of the address information**
 - Two address fields
 - Single address field
 - Variable format

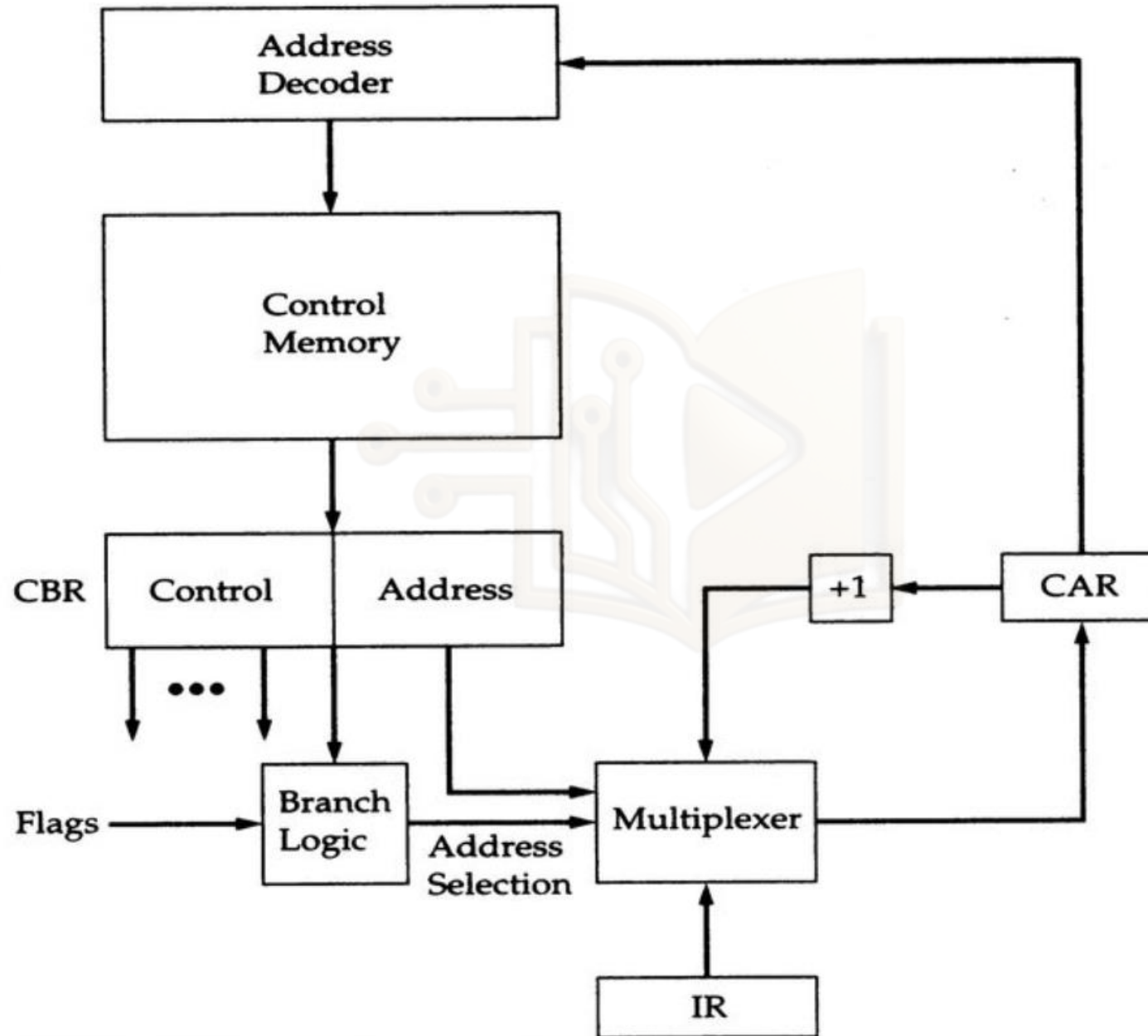
- **The simplest approach is to provide two address fields in each microinstruction.**
- **A multiplexer is provided that serves as a destination for both address fields plus the instruction register.**
- **This scheme requires more bits in the microinstruction than other approaches.**

BCL Two Address Fields



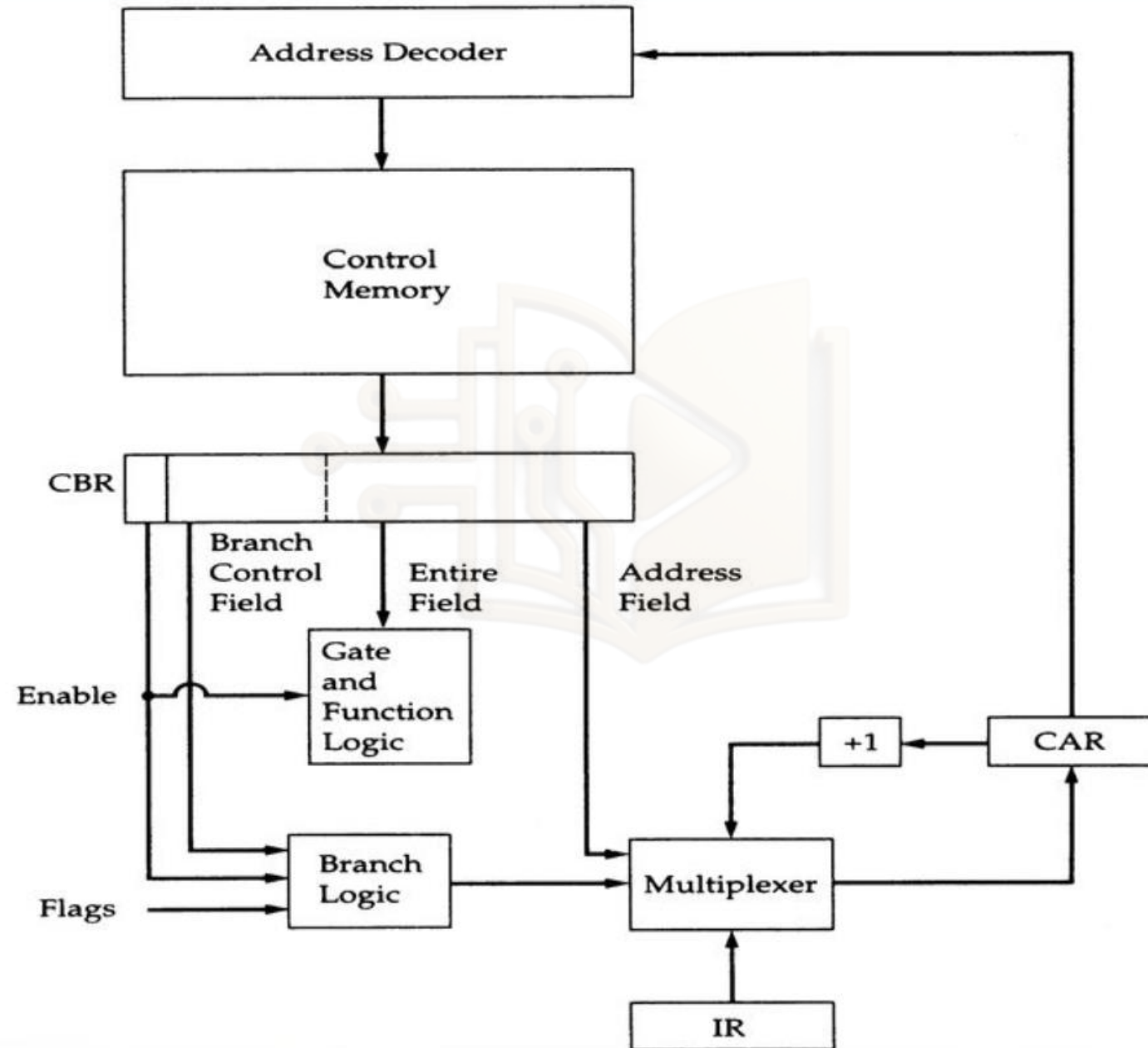
- **A common approach is to have a single address field**
 - Options for next address are as follows
 - Address field
 - Instruction register code
 - Next sequential address
- **The address-selection signals determine which option is selected**
 - Reduces the number of address fields to one.
 - That the address field often will not be used.
 - There is some inefficiency in the microinstruction coding scheme.

BCL Single Address Field



- **Two entirely different microinstruction formats**
 - The first format
 - The next address is either the next sequential address or an address derived from the instruction register.
 - The second format
 - Either a conditional or unconditional branch is being specified
 - One disadvantage of this approach is that one entire cycle is consumed with each branch microinstruction.

BCL Variable Format



Assignment

Write in detail about Micro instruction Execution.

Write the applications of Hardwired and Microprogrammed Control unit.



